

Aarohan Bank Internal GenAI Platform

Workshop-deployable internal systems design using C4 and ADR — from business problem to a governed, auditable, phased GenAI platform.

C4 Levels 1–4

RAG architecture

LLM runtime internals

Governance & controls

47 ADRs

Deployment & operations

Evaluation & observability

Workshop objective

Convert one business problem into a complete, defensible platform architecture.



The case

Aarohan Bank is building an internal GenAI platform for employees, analysts, developers, compliance, customer-service, and operations teams. The bank does not want disconnected chatbots — it wants one governed platform that supports multiple use cases safely.

Policy Q&A · Customer-service assistance · Case summarization ·
Compliance investigation · Operational analytics · Structured extraction ·
Developer assistance · Workflow drafting · Bounded agentic investigation



What participants produce

C4 Context, Container, and Component diagrams
C4 code-level contracts
ADR register and trade-off matrix
Deployment roadmap
Evaluation and observability plan
Final architecture defense

Audience and expected outcomes

Designed for practitioners who must make GenAI real inside a regulated enterprise.



Who this is for

Senior architects · AI platform engineers · GenAI solution architects · Forward deployed engineers · Enterprise architecture teams · Security architects · Risk and compliance teams · Data and platform engineering teams · Engineering leaders · Product and program leaders.



By the end you can

Explain why a simple chatbot fails in a bank Design all four C4 levels for a GenAI platform Choose RAG strategies per question type Reason about LLM runtime internals and cost Write ADRs that survive governance review Defend the architecture end-to-end

Agenda — the learning journey

We move top-down through the architecture, exactly as the source design does.

Business problem → Architecture thinking → C4 context → C4 containers → C4 components → Code-level contracts → RAG architecture → Runtime internals → Governance controls → ADRs → Deployment design → Evaluation & observability → Final architecture defense



Understand

Each section opens with concepts and the reference design from the case study.



Discuss

Facilitator prompts and discussion questions appear throughout — argue the trade-offs.



Apply

Eleven hands-on activities build your team's own version of the architecture.

01

SECTION 1

The business problem

Why Aarohan Bank chose a platform over a pile of chatbots — and the constraints that shape everything downstream.

Purpose of the case study · Phased deployment · Departmental demands · Fragmentation risk · Platform ambition · Business constraints · First production use case

Purpose of the case study

One governed internal platform serving many teams — not disconnected chatbots.



Who the platform serves

Employees, analysts, developers, compliance teams, customer-service teams, and operations teams — internal users only in the first release.



What it must support

Policy Q&A · customer-service assistance · case summarization · compliance investigation · operational analytics · structured extraction · developer assistance · workflow drafting · bounded agentic investigation.



The realism constraint

The bank cannot build everything at once. The architecture must support phased deployment — each phase adds capability on the same governed foundation. The workshop goal: convert this business problem into C4 diagrams, code contracts, an ADR register, a trade-off matrix, a deployment roadmap, an evaluation and observability plan, and a final architecture defense.

Phased deployment ambition

Five phases, one architecture. Later phases must not require redesigning earlier ones.

Phase 1 — Governed policy Q&A

↓ Phase 2 — RAG + tool calling + human approval ↓ Phase 3 — SQL RAG + GraphRAG + structured workflows ↓ Phase 4 — Agentic investigation + optimized LLM runtime ↓
Phase 5 — Enterprise-scale model routing, observability, evals, and FinOps

Six teams, six demands

Every department is already experimenting with GenAI — the demand is real and immediate.



Customer service

Wants a policy assistant.



Compliance

Wants regulation and policy investigation support.



Operations

Wants automatic case summaries.



Data team

Wants natural-language analytics.



Engineering

Wants developer assistance.



Risk

Wants traceable and auditable AI outputs.

The fragmentation risk — and the platform ambition

If every team builds separately, the bank gets ten predictable failures.

duplicate AI tools · unapproved models · inconsistent answers · unmanaged prompts · weak retrieval governance · data leakage risk · uncontrolled costs · poor auditability · no common evals · no consistent rollback path

Build one internal GenAI platform that many business teams can use safely, with shared controls for identity, entitlement, RAG, tools, model routing, workflow, evaluation, observability, cost, and audit. The platform ambition — Aarohan Bank

Workshop business constraints

Realistic enterprise constraints — every design decision must respect all nine.

- **Internal-only** The platform is internal-only in the first release
- **Human approval for high impact** High-impact customer actions require human approval
- **Grounded answers** All production answers must be grounded in approved sources where applicable
- **Governed tools** All tool calls must pass through a governed tool gateway
- **Phased rollout** Production rollout must be phased
- **No autonomous customer contact** Customer-facing autonomous responses are not allowed initially
- **LLM never approves** The LLM must not directly approve fee waivers, loans, refunds, complaints, or regulatory actions
- **Clean context** Unauthorized documents must not enter retrieval or model context
- **Everything versioned** All model, prompt, retrieval, and tool changes must be versioned

The core use case: governed policy Q&A

Employees ask internal policy questions and receive source-grounded answers.

“Can I waive a credit card late-payment fee if the customer paid during the grace period?” The example question every team designs against

Why this is a good workshop case — one innocent question requires nine capabilities:

policy retrieval · latest policy handling · access control · customer data lookup · rules-engine decisioning · human approval · citation validation · structured output · audit trail

It forces participants to design beyond a chatbot.

Recap — the business problem

Four ideas to carry forward.

- **Platform, not chatbots**
Six teams demand GenAI; separate builds produce ten predictable failures. One governed platform with shared controls is the answer.
- **Phases are architecture**
Five deployment phases mean the foundation (identity, audit, registries, evals) must be built for the end state from day one.
- **Constraints generate design**
Nine business constraints — human approval, grounded answers, governed tools, versioning — directly produce the containers we design later.
- **One question, nine capabilities**
The fee-waiver policy question is the thread: every later section shows how a piece of the platform serves this one use case.

02

SECTION 2

Why the simple chatbot design fails

The naive design, its eight failure modes, and the corrected production pipeline that replaces it.

Naive design · Eight failure modes · Corrected 13-step pipeline · Activity 1

The naive design — and its eight failure modes

This is what most GenAI pilots look like. It is not production-ready.

- Employee → Chatbot → LLM → Answer
- ✗ The model may hallucinate policy.
- ✗ The model may use outdated knowledge.
- ✗ The answer may not cite approved sources.
- ✗ The system may ignore entitlement.
- ✗ The system may expose restricted data.
- ✗ The answer may not be auditable.
- ✗ The model may recommend an action it is not authorized to take.
- ✗ There may be no eval, rollback, or incident process.

The corrected production design

Thirteen stages between the user and the answer — each one closes a specific failure gap.

User → Identity & entitlement → Risk tiering → AI gateway → Task routing → Prompt compilation → RAG / tools / rules → Model routing → LLM runtime → Validation → Human approval → Evidence package → Audit & observability

Before the model

Identity, entitlement, risk tiering, gateway checks, task routing, and prompt compilation decide whether and how the model is called.

Around the model

RAG supplies evidence, tools supply facts, rules supply deterministic decisions, and the router picks the right model.

After the model

Validation, human approval, evidence packaging, and audit make the output safe, accountable, and traceable.

Activity 1 — Break the chatbot

ACTIVITY 1 · 15 MINUTES

Goal

- Identify why the simple chatbot architecture fails for the fee-waiver policy question.

Instructions

- Take the naive Employee → Chatbot → LLM → Answer design.
- Walk the fee-waiver question through it step by step.
- List every failure your table finds; tag each as data, control, or accountability.
- Map each failure to the pipeline stage that would fix it.

Expected output

- A failure list (aim for all eight) with the corrective stage for each.

Evaluation criteria

Coverage of the eight failure modes

Correct failure-to-stage mapping

- Banking-specific reasoning, not generic "AI is risky"

Facilitator notes

- Push tables past hallucination — entitlement, auditability, and unauthorized action are the ones teams miss. If a table finishes early, ask which failures a better model alone would not fix.

Recap — from chatbot to platform

Demos hide system failures

Eight failure modes — hallucination, staleness, missing citations, ignored entitlement, data exposure, no audit, unauthorized action, no ops process.

- **The model is one box of thirteen**

The corrected pipeline surrounds the LLM with controls before, around, and after generation.

- **Failures are requirements**

Each failure names a platform capability we now design layer by layer, starting with C4 Level 1.

03

SECTION 3

C4 Level 1 — System context

The platform as one box: who uses it, which enterprise systems it touches, and where the boundary sits.

System in scope · 9 primary users · 17 external systems · Context diagram · Activity 2

System in scope

One system, one purpose, one hard limit.

System name

Aarohan Bank internal GenAI platform

Purpose

Provide governed GenAI capabilities to internal bank users.

Capabilities

Language, retrieval, reasoning, tool use, workflow drafting, and investigation support.

The platform does not own final regulated decisions. The scope limit that shapes every downstream control

Primary users

Nine roles — five ask, one approves, three govern and operate.

USER	NEED
Customer-service employee	Ask policy questions and draft customer responses
Compliance analyst	Investigate policies, exceptions, and operational patterns
Operations user	Summarize cases and prepare workflow updates
Data analyst	Ask operational analytics questions
Developer	Search code, generate tests, review technical docs
Supervisor	Review and approve AI recommendations
Risk/compliance owner	Define controls, evidence, and approval rules
Platform operator	Run models, monitor cost, latency, quality, and incidents
Internal auditor	Review traceability, evidence, and decision logs

External systems — 17 integrations in four groups

The platform orchestrates the enterprise; it does not replace it.

Identity & access

Identity provider — authentication · HR / role system — role, department, business unit · Entitlement / IAM — access rights · Consent / privacy system — purpose and data-boundary checks

Knowledge & data

Policy document repository — policies, SOPs, circulars · SharePoint / document stores — internal documents · Data warehouse / lakehouse — structured analytics · Semantic layer — governed business metrics

Business systems

CRM — customer information · Case-management system — complaints, service requests, case notes · Core banking systems — account, payment, fee, transaction data · Rules engine — deterministic eligibility and thresholds · Workflow engine — approval and case routing · Ticketing / engineering tools — developer and IT workflows

AI & assurance

Model runtime / model providers — LLMs, SLMs, embedding models, rerankers · Observability platform — metrics, logs, traces · Audit store — evidence, decisions, versions, approvals

C4 Level 1 — System context diagram

Internal users on top, one platform in the middle, the enterprise estate around it.

Internal users — service, compliance, ops, data, dev, supervisors,
risk, operators, audit ↓ AAROHAN BANK INTERNAL GENAI
PLATFORM governed GenAI capabilities · no ownership of regulated
decisions ⇅

Identity provider · HR / role system · Entitlement service ·
Consent & privacy · Policy repositories · Document stores ·
CRM · Case management · Core banking · Warehouse +
semantic layer · Rules engine · Workflow engine ·
Engineering tools · Model runtime · Observability platform ·
Audit store

Activity 2 — Draw the C4 Level 1 context diagram

ACTIVITY 2 · 20 MINUTES

Goal

- Define the system boundary for the platform.

Instructions

- Draw the platform as a single system.
- Place all user types who interact with it.
- Place external systems: identity, data sources, business systems, model systems, audit stakeholders.
- Label every interaction with what flows across it.

Expected output

- One C4 Level 1 context diagram per team.

Evaluation criteria

Clear single-system boundary

All nine user roles represented

- Identity, knowledge, business, and assurance systems all present
- No internal components leaking into the diagram

Facilitator notes

- Watch for missing governance actors (auditor, risk owner) and missing assurance systems (audit store, observability). Ask "who reads the logs?" to surface them.

Recap — system context

One box, hard boundary

The platform provides language, retrieval, reasoning, tool use, and investigation support — and never owns regulated decisions.



Governance users are users

Supervisors, risk owners, operators, and auditors appear at L1 with equal standing to the teams asking questions.



17 external systems

Identity & access, knowledge & data, business systems, AI & assurance — the platform orchestrates, it doesn't replace.

04

SECTION 4

C4 Level 2 — Container architecture

Inside the platform: 23 deployable containers in three planes — experience & control, knowledge & action, model & assurance.

23 containers · Responsibilities · Container diagram · Activity 3

Containers — experience & control plane

The front door and the decision-makers: who are you, what may you do, what are you really asking?

CONTAINER	RESPONSIBILITY
Internal AI Portal / API	User experience, chat, workflow forms, streaming output
Identity, Entitlement, Consent & Risk Service	User context, access rights, data boundary, risk tier
AI Gateway	Prompt policy, DLP, prompt-injection checks, rate limits
Task Router	Classifies intent, domain, retrieval need, tool need, schema need
Prompt Compiler	Builds model-ready prompt with evidence, tools, schema, and token budget
Context Orchestrator	Coordinates RAG, SQL RAG, GraphRAG, tools, memory, and cache

Containers — knowledge & action plane

Everything that grounds the model in facts or lets it touch enterprise systems — under policy.

CONTAINER	RESPONSIBILITY
RAG Platform	Hybrid retrieval, ACL retrieval, temporal retrieval, reranking, context packing
RAG Ingestion Pipeline	Parsing, chunking, metadata, embeddings, indexing, corpus registry
GraphRAG Service	Entity and relationship retrieval for impact analysis
SQL RAG Service	Governed natural-language analytics through semantic layer
Tool Gateway	Approved enterprise tool execution
Rules Integration Service	Calls deterministic rules engine
Workflow & Human Approval Service	Human review, approval, escalation, attestation
Agent Runtime	Bounded investigation workflows
Sandbox Execution Service	Safe code, file, SQL, and analytics execution

Containers — model & assurance plane

Serving the model efficiently — and proving every answer was safe.

CONTAINER	RESPONSIBILITY
Model Router	Chooses model by task, risk, latency, cost, and sensitivity
LLM Serving Runtime	Tokenization, prefill, KV cache, decode, batching, quantization, streaming
Cache Service	Prefix, retrieval, semantic, response, and tool-result caches
Validation & Evidence Service	Schema, citation, faithfulness, policy, and approval checks
Evaluation Service	Golden datasets, regression tests, RAG evals, safety evals
Observability & FinOps Service	Traces, metrics, GPU usage, token cost, alerts
Audit Store	Immutable evidence and decision history
Registry Services	Prompt, model, adapter, corpus, schema, tool, and index registries

C4 Level 2 — Container diagram

The request path top to bottom; the context orchestrator fans out to knowledge and action services.



User

↓ Internal AI Portal / API ↓ Identity, Entitlement, Consent & Risk Service ↓ AI Gateway ↓ Task Router ↓ Prompt Compiler ↓ Context Orchestrator ↓ Model Router ↓ LLM Serving Runtime ↓ Validation & Evidence Service ↓ Workflow & Human Approval Service ↓ Output / Action Layer ↓ Audit Store + Observability + Evaluation + FinOps



Context Orchestrator fans out to

RAG Platform — hybrid retrieval · ACL-aware retrieval · temporal retrieval · reranker · context packer



GraphRAG Service

SQL RAG Service



Tool Gateway

Rules Integration Service



Cache Service

Agent Runtime / Sandbox where required



Reading the diagram

The spine is the corrected pipeline from Section 2, made deployable. Every request descends the spine; only the orchestrator branch varies by task. Audit, observability, evaluation, and FinOps receive events from every layer.

Activity 3 — Draw the C4 Level 2 container diagram

ACTIVITY 3 · 30 MINUTES

Goal

- Decompose the platform into deployable containers with clear responsibilities and boundaries.

Instructions

- Identify the major deployable containers.
- Write a one-line responsibility for each.
- Draw data/control flow between containers.
- Mark security boundaries and data boundaries.

Expected output

- A C4 Level 2 container diagram with responsibilities and boundaries.

Evaluation criteria

- Containers are deployable units, not concepts
- Control plane precedes the model in the flow
- Assurance containers (validation, evals, observability, audit) present

Security/data boundaries marked

Facilitator notes

- Teams typically forget the cache service, registries, and the human-approval service. Ask: "where does an approved answer's evidence live five years from now?"

Recap — container architecture

Three planes, 23 containers

Experience & control · knowledge & action · model & assurance. Each is deployable, scalable, and auditable on its own.

- **One spine, variable fan-out**

Every request walks the same governed spine; the context orchestrator varies what evidence and actions are gathered.

- **Assurance is built in**

Validation, evaluation, observability, audit, and seven registries are containers — not process documents.

05

SECTION 5

C4 Level 3 — Component architecture

Opening the containers: 18 component sets covering identity, gateway, routing, prompt compilation, all four RAG services, tools, rules, approval, agents, sandbox, model routing, LLM runtime, validation, evaluation, and observability.

Identity & risk · AI Gateway · Task Router · Prompt Compiler · RAG ingestion + retrieval · GraphRAG · SQL RAG · Tool Gateway · Rules · Approval · Agents · Sandbox · Model Router · LLM Runtime · Validation · Evals · Observability

Identity, entitlement, consent & risk components

Seven components that turn "a login" into a full access context.



Identity Resolver

Resolves authenticated user



Role Resolver

Gets role, business unit, region



Entitlement Resolver

Determines allowed corpus, tools, and data



Consent Checker

Checks purpose and consent rules



Data Boundary Checker

Checks geography and restricted data boundaries



Risk Classifier

Classifies request as low, medium, or high risk



Access Context Builder

Creates downstream access context

Access context = user + role + region + allowed data + allowed tools + risk tier

AI Gateway components

The security checkpoint: what is in this prompt, and is it allowed?



Input DLP Scanner

Detects sensitive data and secrets



Prompt Injection Detector

Detects malicious instructions



Request Policy Engine

Applies enterprise AI use policy



Rate Limiter

Enforces user and tenant quotas



Redaction Service

Masks restricted data where required



Gateway Decision Engine

Allows, blocks, redacts, or escalates request

Incoming request → DLP + injection +
policy + quota checks → Allow · Block ·
Redact · Escalate

Task Router components

Seven classifiers that turn a free-text question into an execution plan.



Intent Classifier

Policy Q&A, analytics, code help, case summary, investigation



Domain Classifier

Cards, lending, wealth, fraud, compliance, IT



Retrieval Need Classifier

Selects document RAG, SQL RAG, GraphRAG, tools, or no retrieval



Tool Need Classifier

Determines required enterprise tools



Output Schema Selector

Selects structured output schema where required



Human Approval Classifier

Determines approval requirement



Orchestration Planner

Produces an execution plan

Prompt Compiler components

Ten components that assemble the model-ready prompt — deterministically, under a token budget.

 **Chat Template Renderer** Applies model-specific chat format

 **Role Instruction Injector** Adds user role and business-unit instructions

 **Tool Result Inserter** Adds approved tool outputs

 **Schema Injector** Adds JSON schema or tool schema

 **Context Compressor** Compresses low-priority history or evidence

 Prompt compilation is context engineering. It is not casual prompt writing. Workshop teaching point

 **System Instruction Assembler** Adds bank AI policy and behavior rules

 **Evidence Inserter** Adds retrieved evidence

 **Rules Result Inserter** Adds deterministic decision trace

 **Token Budget Manager** Allocates token budget

 **Prompt Version Tagger** Records prompt version

RAG Ingestion Pipeline components

Fifteen components between "a document exists" and "a governed corpus entry exists".

● Connect → Classify → Validate owner → Parse + OCR → Clean + tables → Chunk → Enrich metadata → Embed + index → Register + QA

● **Document Classifier** Identifies document type and sensitivity

● **Parser** Extracts text and structure

● **Layout Cleaner** Removes headers, footers, noise

● **Chunker** Splits content using document-aware strategy

● **Embedding Generator** Creates dense vectors

● **Vector Indexer** Builds vector index

● **Corpus Registry Updater** Records approved corpus

● **Source Connector** Connects to policy repositories and document stores

● **Ownership Validator** Ensures document owner exists

● **OCR Processor** Handles scanned documents

● **Table Extractor** Preserves tables

● **Metadata Enricher** Adds owner, version, region, access group, effective date

● **Keyword Indexer** Builds lexical index

● **Graph Extractor** Extracts entities and relationships where needed

● The fifteenth component — the Quality Checker — checks metadata, lineage, and chunk quality before anything goes live.

RAG Platform components — the retrieval pipeline

Fourteen components from query to packed, cited, answerable evidence.

- **Retrieval Strategy Selector** Chooses RAG type
 - **Temporal Filter** Selects current effective documents
 - **Keyword Retriever** Exact search
 - **Rank Fusion Engine** Combines keyword and vector results
 - **Deduplicator** Removes repeated chunks
 - **Context Packer** Fits evidence into token budget
 - **Answerability Checker** Decides if evidence is sufficient
 - **ACL Filter** Applies entitlement before retrieval
 - **Metadata Filter** Filters by product, region, classification
 - **Vector Retriever** Semantic search
 - **Reranker** Re-scores candidate chunks
 - **Authority Ranker** Prefers official approved sources
 - **Citation Anchor Builder** Preserves source references
 - **Contradiction Detector** Detects conflicting sources
- Filter first ACL · temporal · metadata → Retrieve hybrid keyword + vector + fusion →
Refine rerank · dedupe · authority → Assemble pack · cite · answerability · contradictions

GraphRAG components

Entity-and-relationship retrieval for questions that cross documents.



Entity Extractor

Extracts policies, products, systems, roles, fields



Relationship Extractor

Extracts affects, depends-on, approves, owns, supersedes



Graph Builder

Builds knowledge graph



Community Summarizer

Summarizes graph communities



Graph Query Planner

Plans graph traversal



Graph Retriever

Retrieves connected evidence



Graph Evidence Assembler

Combines graph paths and source documents



Use GraphRAG when the question is about

impact · lineage · dependency · ownership · relationships · cross-document reasoning "If we change the late-fee policy, which products, SOPs, and teams are affected?" — vector search over chunks cannot answer this; a graph of affects/depends-on/supersedes edges can.

SQL RAG components

Governed natural-language analytics — through the semantic layer, never raw tables.

- **Metric Resolver** Maps business terms to approved metrics
 - **Semantic Layer Adapter** Connects to governed metric model
 - **NL-to-SQL Generator** Produces SQL candidate
 - **SQL Validator** Checks syntax and allowed operations
 - **Row-Level Security Enforcer** Applies data access
 - **Query Cost Guard** Prevents expensive or unsafe queries
 - **Result Summarizer** Explains returned result
 - **Data Lineage Tracker** Records tables and metrics used
- Use RAG for documents. Use SQL RAG for structured facts. The retrieval-selection rule

Tool Gateway components

Nine components that stand between "the model wants" and "the enterprise does".

- **Tool Registry** Stores approved tools and schemas
- **Credential Broker** Provides scoped credentials
- **Idempotency Manager** Prevents duplicate writes
- **Tool Result Redactor** Removes restricted fields
- **Circuit Breaker** Disables failing or unsafe tools
- **Tool Policy Engine** Checks whether the tool is allowed
- **Tool Executor** Calls enterprise APIs
- **Read/Write Separator** Separates lookup tools from action tools
- **Tool Audit Logger** Logs tool call and result reference
- The model can request a tool. The platform decides whether the tool runs. The tool-governance rule

Rules Integration components

Five components that keep eligibility, thresholds, and exceptions deterministic.



Rule Package Resolver

Selects rule version



Rule Input Builder

Converts facts into rule inputs



Rule Engine Adapter

Calls deterministic rules engine



Decision Trace Builder

Records rule path



Rule Result Validator

Checks completeness

Rules decide. Models explain. The decisioning rule

Workflow & Human Approval components

Where accountability lives: seven components for review, attestation, and escalation.



Approval Queue

Holds cases needing review



Reviewer Assignment Engine

Assigns supervisor or compliance reviewer



Approval UI

Shows recommendation, evidence, rule trace, tool outputs



Attestation Recorder

Records approve, edit, reject, escalate



Escalation Router

Sends complex cases to higher authority



Workflow Adapter

Updates case or workflow system



Notification Service

Notifies approvers and requesters

Agent Runtime components

Bounded investigation — autonomy inside hard limits.

● **Planner** Breaks investigation into steps

● **Tool Selector** Selects allowed tools

● **Verifier** Checks intermediate outputs

● **Human Escalation Trigger** Escalates uncertain results

● **Agent boundaries** ✓ read-only by default ✓ approved tools only ✓ step limits ✓ cost limits ✓ no irreversible action ✓ human review required

● **State Store** Stores intermediate evidence

● **Budget Controller** Enforces token, step, time, and cost limits

● **Stop Condition Evaluator** Stops when complete or unsafe

● **Agent Trace Logger** Logs every step

Sandbox Execution components

Nine components for running generated code, SQL, and file processing safely.

-  **Code Runner** Executes generated code safely
-  **SQL Sandbox** Tests generated queries
-  **File Processor** Handles uploaded files safely
-  **Package Allowlist Manager** Controls libraries
-  **Network Guard** Blocks unsafe network access
-  **Timeout Controller** Stops long-running jobs
-  **Secret Scanner** Prevents credential leakage
-  **Artifact Validator** Checks generated outputs
-  **Sandbox Audit Logger** Logs execution

Model Router components

Seven components that choose the right model per request — not the biggest model for everything.



Model Registry Reader

Reads approved models



Routing Policy Engine

Selects model by task and risk



Capability Matcher

Matches context, tools, schema, reasoning need



Cost Estimator

Estimates model cost



Latency Estimator

Estimates response time



Private Model Selector

Chooses private model for sensitive data



Fallback Selector

Chooses fallback model

LLM Serving Runtime components

Fifteen components — the core loop plus the optimizations that make GPU serving affordable.

● Tokenizer → Prefill Engine → KV Cache → Decode Engine → Streaming Handler

● **Prefill Engine** Processes input prompt

● **KV Cache Manager** Stores attention keys and values

● **Prefix Cache Manager** Reuses common prompt prefixes

● **Sampler** Controls decoding

● **Speculative Decoder** Uses draft and verifier model

● **Quantized Runtime Loader** Loads quantized variants

● **Streaming Handler** Streams tokens

● **Tokenizer** Converts prompt to tokens

● **Decode Engine** Generates tokens

● **Paged KV Allocator** Manages KV memory blocks

● **Scheduler** Performs continuous batching

● **Guided Decoder** Enforces schema or grammar

● **Attention Kernel Layer** Optimized attention execution

● **Adapter Loader** Loads LoRA/adapters

● **GPU Telemetry Collector** Captures runtime metrics

Validation & Evidence components

Eight checks between "the model said" and "the user sees" — plus the evidence that outlives both.



Schema Validator

Checks structured output



Citation Validator

Checks citations support claims



Faithfulness Checker

Checks answer against evidence



Policy Output Checker

Checks disallowed outputs



Tool Result Consistency Checker

Checks text against tool results



Human Approval Requirement Checker

Ensures approval is triggered



Evidence Packager

Packages prompt, sources, model, tools, rules, decisions



Audit Event Writer

Writes immutable audit record

Evaluation components

Nine components that decide whether any change is safe to release.

-  **Golden Dataset Store** Stores test cases
-  **Model Eval Runner** Tests answer quality
-  **Agent Eval Runner** Tests agent boundaries
-  **Regression Gate** Blocks unsafe releases
-  **Eval Dashboard** Shows quality trends
-  **RAG Eval Runner** Tests retrieval recall and citation quality
-  **Tool Eval Runner** Tests tool-call correctness
-  **Security Eval Runner** Tests prompt injection and data leakage
-  **Human Review Sampler** Samples production outputs

Observability & FinOps components

Nine components that make the platform operable — and its cost visible.

-  **Trace Collector** Captures end-to-end trace
-  **Metrics Aggregator** Aggregates latency, cost, token, GPU metrics
-  **Cost Calculator** Computes cost per request, model, tenant
-  **Alert Manager** Alerts on failure, latency, cost, safety
-  **GPU Monitor** Tracks model-serving utilization
-  **Cache Hit Monitor** Tracks cache performance
-  **Retrieval Quality Monitor** Tracks retrieval failures
-  **Approval Analytics** Tracks approve, edit, reject rates
-  **Incident Dashboard** Tracks incidents and rollback status

Activity 4 — Decompose one container into components

ACTIVITY 4 · 30 MINUTES

Goal

- Produce a C4 Level 3 component diagram for one container your team chooses.

Instructions

- Pick one: AI Gateway, Task Router, Prompt Compiler, RAG Platform, Tool Gateway, Model Router, LLM Runtime, Validation Service, or Observability Service.
- List its components with one-line responsibilities.
- Draw the internal flow between components.
- Mark the container's inputs and outputs.

Expected output

- One C4 Level 3 component diagram with named responsibilities.

Evaluation criteria

Components have single responsibilities

- Governance components present (not just the happy path)
- Clear input/output contracts to neighboring containers

Facilitator notes

- Assign different containers to different tables so the debrief assembles the full platform. Push each table: "which component fails first, and who notices?"

Recap — component architecture

Control before the model

Identity builds the access context; the gateway allows/blocks/redacts/escalates; the router plans; the compiler assembles versioned, budgeted context.



Evidence over memory

Ingestion governs the corpus; retrieval filters before searching; GraphRAG serves relationships; SQL RAG serves structured facts through the semantic layer.



Action under policy

Tool gateway: the model requests, the platform decides. Rules decide, models explain. Approval attests. Agents are bounded. Code runs in a sandbox.



Serving plus assurance

The router picks models; the runtime optimizes the tokenize→prefill→KV→decode loop; validation, evals, and observability prove it all works.

06

SECTION 6

C4 Level 4 — Code-level contracts

Not source code — module-level contracts, interfaces, and schemas that make the components interoperable and auditable.

Request & access context · Risk & orchestration plan · Document · chunk · retrieval · Tool call · Prompt package · Rule & routing decisions · Validation
· evidence · trace

The contract map

Fourteen typed objects trace one request from arrival to audit.

GenAIRequest → AccessContext → RiskClassification → OrchestrationPlan → RetrievalResult over Chunk + DocumentMetadata → ToolCallRequest / Result → RuleDecision → PromptPackage → ModelRoutingDecision → ValidationResult → EvidencePackage + RuntimeTrace



Why contracts matter

Every arrow in the C4 L2 and L3 diagrams carries one of these objects. Versioned contracts are what let teams build containers independently, let validators check outputs mechanically, and let auditors reconstruct any answer years later.

GenAI request & access context

What arrives — and what identity turns it into.

contract.ts

```
type GenAIRequest = {
  requestId: string; tenantId: string; userId: string;
  userQuestion: string;
  channel: "portal" | "api" | "workflow" | "batch";
  sessionId?: string;
  uploadedFiles?: FileRef[];
  requestedAction?: string;
  metadata: { timestamp: string; locale?: string;
    businessUnit?: string; region?: string; };
};

type AccessContext = {
  userId: string; tenantId: string; roles: string[];
  businessUnit: string; region: string;
  allowedCorpusIds: string[];
  allowedToolIds: string[];
  allowedDataScopes: string[];
  restrictedActions: string[];
  dataBoundary: { country: string;
    crossBorderAllowed: boolean;
    sensitiveDataAllowed: boolean; };
  approvalRequiredByDefault: boolean;
};
```

Risk classification & orchestration plan

Risk is typed; the execution plan is explicit.

contract.ts

```
type RiskClassification = {
  riskTier: "low" | "medium" | "high";
  customerImpacting: boolean;
  financialImpacting: boolean;
  regulatedDecision: boolean;
  externalCommunication: boolean;
  requiresHumanApproval: boolean;
  explanation: string;
};

type OrchestrationPlan = {
  intent: "policy_qa" | "case_summary" | "customer_support"
    | "analytics" | "code_assist"
    | "compliance_investigation" | "workflow_action";
  retrievalPlan: { useRAG: boolean;
    ragTypes: Array<"hybrid" | "acl_aware" | "temporal"
      | "graph" | "sql" | "corrective" | "agentic">; };
  toolPlan: { requiredTools: string[];
    writeActionRequested: boolean; };
  modelPlan: { preferredModelClass: "small" | "large"
    | "reasoning" | "private" | "fine_tuned";
    structuredOutputRequired: boolean; };
  approvalPlan: { humanApprovalRequired: boolean;
    approverRole?: string; };
};
```

Document metadata & chunk object

Governance travels with the content — every chunk knows its provenance.

`contract.ts`

```
type DocumentMetadata = {
  documentId: string; title: string;
  sourceSystem: string; owner: string;
  businessUnit: string; region: string; product?: string;
  classification: "public_internal" | "confidential"
    | "restricted";
  accessGroups: string[];
  version: string;
  approvalStatus: "draft" | "approved" | "retired";
  effectiveDate?: string; expiryDate?: string;
  supersededBy?: string;
  sourceUrl: string; hash: string;
};

type Chunk = {
  chunkId: string; documentId: string;
  parentSectionId?: string;
  text: string;
  chunkType: "policy_clause" | "faq_pair" | "table"
    | "manual_section" | "legal_clause" | "code_block";
  tokenCount: number;
  metadata: DocumentMetadata;
  embeddingModelVersion: string;
  indexVersion: string;
  citationAnchor: { section?: string; page?: number;
    paragraph?: number; lineRange?: string; };
};
```

Retrieval result

Scored candidates plus an explicit answerability verdict.

Six scores per candidate

Lexical, vector, fusion, rerank, authority, freshness — each retrieval stage leaves its score on the candidate, so ranking decisions are debuggable. The answerability gate Retrieval can say "not answerable, low confidence, here's why" — the platform then refuses or escalates instead of generating a guess (ADR-020).

`contract.ts`

```
type RetrievalResult = {
  query: string;
  retrievalStrategy: string[];
  candidates: Array<{
    chunkId: string; documentId: string; text: string;
    lexicalScore?: number; vectorScore?: number;
    fusionScore?: number; rerankScore?: number;
    authorityScore?: number; freshnessScore?: number;
    citationAnchor: Chunk["citationAnchor"];
  }>;
  answerability: {
    answerable: boolean;
    confidence: "low" | "medium" | "high";
    reason?: string;
  };
};
```

Tool call contract

Request and result — with access context, idempotency, and audit built in.

Carries its permissions

The full `AccessContext` rides along — the gateway re-checks entitlement at execution time.

Safe to retry, safe to test

`idempotencyKey` prevents double writes; `dryRun` rehearses an action without executing it.

Four outcomes, always audited

`success` · `denied` · `failed` · `requires_approval` — and every result names its `auditEventId`.

contract.ts

```
type ToolCallRequest = {
  toolId: string;
  requestId: string;
  userId: string;
  tenantId: string;
  parameters: Record<string, unknown>;
  accessContext: AccessContext;
  idempotencyKey?: string;
  dryRun: boolean;
};

type ToolCallResult = {
  toolId: string;
  status: "success" | "denied" | "failed"
    | "requires_approval";
  resultRef?: string;
  redactedResult?: Record<string, unknown>;
  errorCode?: string;
  auditEventId: string;
};
```

Prompt package

The compiler's output: everything the model sees, versioned and budgeted.

Reading the package

Instructions, question, evidence (with citation anchors), tool facts, the rule decision, and the output schema — assembled under an explicit token budget with reserved output space. Because the package records `promptTemplateVersion` and `modelId`, any production answer can be reproduced exactly during audit or incident review.

`contract.ts`

```
type PromptPackage = {
  promptId: string;
  promptTemplateVersion: string;
  modelId: string;
  systemInstructions: string;
  userQuestion: string;
  evidenceBlocks: Array<{
    evidenceId: string; chunkId: string; text: string;
    citationAnchor: Chunk["citationAnchor"];
  }>;
  toolResults: ToolCallResult[];
  rulesResult?: RuleDecision;
  outputSchema?: JsonSchema;
  tokenBudget: {
    maxInputTokens: number;
    reservedOutputTokens: number;
    evidenceTokens: number;
    historyTokens: number;
  };
};
```

Rule decision & model routing decision

Deterministic outcomes and model choices — both fully explained.

`contract.ts`

```
type RuleDecision = {
  rulePackageId: string;
  ruleVersion: string;
  decision: "eligible" | "not_eligible"
    | "requires_supervisor_review"
    | "insufficient_information";
  reasonCodes: string[];
  decisionTrace: Array<{
    ruleId: string;
    input: string;
    result: string;
  }>;
};

type ModelRoutingDecision = {
  selectedModelId: string;
  selectedModelClass: "small" | "large" | "reasoning"
    | "private" | "fine_tuned";
  fallbackModelId: string;
  reason: string;
  expectedCostClass: "low" | "medium" | "high";
  expectedLatencyClass: "low" | "medium" | "high";
  quantizedVariant?: "none" | "int8" | "int4" | "fp8";
  adapterId?: string;
};
```

Validation result & evidence package

The safety verdict — and the immutable record that outlives the request.

`contract.ts`

```
type ValidationResult = {
  schemaValid: boolean;
  citationsValid: boolean;
  faithfulToEvidence: boolean;
  policyCompliant: boolean;
  humanApprovalRequired: boolean;
  blockingIssues: string[];
  warnings: string[];
};

type EvidencePackage = {
  requestId: string;  userId: string;  tenantId: string;
  riskTier: string;
  modelId: string;  modelVersion: string;
  promptTemplateVersion: string;
  embeddingModelVersion?: string;  indexVersion?: string;
  toolCallIds: string[];
  ruleDecisionId?: string;
  citedSources: Array<{ documentId: string; chunkId: string;
    citationAnchor: Chunk["citationAnchor"]; }>;
  validationResult: ValidationResult;
  humanDecision?: { reviewerId: string;
    decision: "approved" | "rejected" | "edited" | "escalated";
    reason?: string;  timestamp: string; };
};
```

Runtime trace

One object connects governance to GPU economics.

What it measures

ttftMs — time to first token (user-perceived responsiveness). tpotMs — time per output token (streaming speed). Prefill and decode latency are tracked separately (ADR-027) because they stress different resources. Three cache-hit flags and kvCacheBytes make the runtime optimizations of Section 5 measurable per request — and costEstimate per tenant funds the FinOps conversation.

contract.ts

```
type RuntimeTrace = {
  requestId: string; tenantId: string; modelId: string;
  inputTokens: number; outputTokens: number;
  ttftMs: number; tpotMs: number;
  totalLatencyMs: number;
  prefillLatencyMs: number;
  decodeLatencyMs: number;
  kvCacheBytes: number;
  prefixCacheHit: boolean;
  retrievalCacheHit: boolean;
  semanticCacheHit: boolean;
  gpuUtilization?: number;
  costEstimate: number;
};
```

Activity 5 — Define code-level contracts

ACTIVITY 5 · 30 MINUTES

Goal

- Write the typed contracts your components exchange.

Instructions

- Define contracts for: request, access context, risk classification, orchestration plan, chunk metadata, retrieval result, tool call, prompt package, validation result, evidence package, runtime trace.
- For each: fields, enums, and which component produces/consumes it.
- Mark which fields exist purely for audit.

Expected output

- C4 Level 4 code contracts (type definitions or schema sketches).

Evaluation criteria

- Contracts cover the full request lifecycle

Versions and IDs support reproducibility

- Enums constrain risky values (status, tier, decision)

Facilitator notes

- Keep teams at module level — no source code. The test question: "could an auditor reconstruct an answer from these objects alone?"

Recap — code-level contracts

Contracts, not code

C4 L4 here means typed objects on the arrows — fourteen of them from GenAIRequest to RuntimeTrace.



Governance as fields

ACLs, versions, hashes, reason codes, decision traces, attestations — every control from earlier sections shows up as a field.



Reproducibility by design

Model, prompt, embedding, and index versions travel with every answer; the evidence package makes audits a query, not an investigation.

07

SECTION 7

Workshop sequence flows

Four end-to-end journeys: policy Q&A, fee-waiver support, compliance investigation, and analytics sandbox.

Policy Q&A · Fee-waiver support · Compliance investigation · Analytics sandbox · Activities 6–8

Policy Q&A flow

The phase 1 flow — retrieval-grounded answering with validation and audit, no tools, no approval queue.

User → AI Portal → Identity / Entitlement / Risk Service → AI Gateway → Task Router → RAG Platform → Prompt Compiler → Model Router → LLM Runtime → Validation & Evidence Service → AI Portal → Audit Store



What to notice

Even the simplest flow makes twelve stops end to end. The answer returns to the portal only after validation, and the audit store receives the evidence package regardless of outcome. This is the minimum governed path — nothing optional was removed.

Fee-waiver support flow

Policy + customer data + deterministic rules + human approval — the full phase 2 pattern.

User → AI Portal → Identity / Entitlement / Risk → AI Gateway → Task Router
→ RAG Platform policy → Tool Gateway payment & waiver history → Rules
Integration Service → Prompt Compiler → Model Router → LLM Runtime →
Validation & Evidence → Human Approval Service → Workflow Engine → Audit
Store



What to notice

Three services join the phase 1 flow: the tool gateway fetches payment and waiver history, the rules engine decides eligibility deterministically, and a human approves before the workflow engine acts. The LLM drafts and explains — it never decides the waiver.

Compliance investigation flow

The phase 4 pattern: a bounded agent orchestrates all three RAG types and tools — then lands at human review.

Compliance Analyst → AI Portal → Identity / Risk Service → AI Gateway →
Agent Runtime → RAG Platform → GraphRAG Service → SQL RAG Service →
Tool Gateway → Agent Verifier → Human Review → Evidence Package →
Audit Store



What to notice

The agent runtime sits inside the governed perimeter — identity and gateway run first, the verifier checks intermediate outputs, and human review is mandatory before the evidence package closes. All three retrieval services combine: documents for policy, graph for relationships, SQL for operational facts.

Analytics sandbox flow

Natural-language analytics with generated SQL — executed only in the sandbox.

Data Analyst → AI Portal → AI Gateway → Task Router → SQL RAG Service → Sandbox Execution Service → Result Summarizer → Validation Service → AI Portal → Audit Store



What to notice

Generated SQL is tested in the sandbox before results reach the analyst; the result summarizer explains what came back, validation checks it, and lineage lands in audit. The model never queries the warehouse directly.

Activity 6 — Select the RAG strategy per question

ACTIVITY 6 · 20 MINUTES

Goal

- Choose the right retrieval strategy for different question shapes.

Instructions

- For each question type, pick from: hybrid, ACL-aware, temporal, GraphRAG, SQL RAG, corrective, agentic — or no retrieval.
- Questions: a policy lookup · "which SOPs does this circular affect?" · "how many waivers last quarter?" · a code-help request · a multi-step investigation.
- Justify each choice in one sentence.

Expected output

- A question-to-strategy matrix with justifications.

Evaluation criteria

- Documents → RAG; structured facts → SQL RAG; relationships → GraphRAG
- ACL and temporal filters applied everywhere applicable

Recognizes when retrieval is unnecessary

Facilitator notes

- The trap: teams pick GraphRAG for everything impressive-sounding. Enforce ADR-017 — graph only for relationship-heavy questions.

Activity 7 — Decide where tool calling is allowed

ACTIVITY 7 · 20 MINUTES

Goal

- Draw the tool-permission boundary for the platform.

Instructions

- List candidate tools: CRM lookup, payment history, waiver history, case update, ticket creation, workflow submission.
- Classify each: read or write.
- Decide per tool: allowed freely, allowed with approval, dry-run only, or denied — per risk tier.
- Define what the circuit breaker does when a tool misbehaves.

Expected output

- A tool-permission matrix (tool × risk tier → policy).

Evaluation criteria

Read/write separation applied

- Write tools carry idempotency and approval requirements
- Matrix references the access context, not job titles

Facilitator notes

- Anchor on the rule: the model can request a tool; the platform decides whether it runs. Push teams to name the deny cases explicitly.

Activity 8 — Define human approval rules

ACTIVITY 8 · 20 MINUTES

Goal

- Specify exactly when a human must review, and who.

Instructions

- Using RiskClassification fields (customerImpacting, financialImpacting, regulatedDecision, externalCommunication), write approval trigger rules.
- Assign approver roles per trigger (supervisor vs. compliance reviewer).
- Define the escalation path for cases the first reviewer can't decide.
- Specify what the reviewer must see (evidence, rule trace, tool outputs).

Expected output

- A human approval matrix: trigger → approver role → escalation path.

Evaluation criteria

- Fee waivers, loans, refunds, complaints, regulatory actions always require approval

Approvers see evidence, not just answers

- Attestation outcomes (approve/edit/reject/escalate) recorded

Facilitator notes

- Challenge over-approval too: if everything needs review, the platform adds no value. Risk tiering exists so low-risk flows can run without a queue.

Recap — sequence flows

One spine, four journeys

Policy Q&A (retrieval only) → fee waiver (+ tools + rules + approval) → investigation (+ agent + all RAG types) → analytics (+ sandbox). Complexity grows with risk and phase.

 **Constant entry and exit**
Every flow starts with identity and the gateway, and ends with validation and audit — no exceptions, no shortcuts.

 **Decisions stay outside the model**
Rules decide eligibility before generation; humans attest after validation; the workflow engine acts only on attested outcomes.

 **Strategy is per question**
Documents → RAG, relationships → GraphRAG, structured facts → SQL RAG, code and analytics → sandbox. The task router chooses; the flows compose.

08

SECTION 8

Deployment architecture

Environments, security zones, GPU serving pools, and the runtime optimizations that make the platform affordable at scale.

5 environments · 7 deployment zones · 7 model-serving pools · 9 runtime optimizations

Environments and deployment zones

Five environments for promotion; seven zones for isolation.

ZONE	CONTAINS
User access zone	AI portal, API edge
Security control zone	AI gateway, DLP, prompt-injection checks
Data access zone	RAG, SQL RAG, GraphRAG, tool gateway
Model serving zone	Model router, LLM serving runtime, GPU pools
Sandbox zone	Isolated code/file/SQL execution
Governance zone	Registries, evaluation, audit, evidence
Observability zone	Logs, metrics, traces, dashboards, alerts

Development → Integration → Staging → Production → Regulated production zone

Model-serving pools

Seven pools — capacity organized by workload shape, not by team.



Small-model pool

Cheap, fast tasks — classification, routing, summaries.



Large-model pool

General high-quality generation.



Reasoning-model pool

Complex multi-step reasoning workloads.



Private-model pool

Sensitive data pinned to in-house serving.



Long-context pool

Big-document and long-history workloads.



Batch-processing pool

Offline summarization and extraction at scale.



Sandbox-assisted pool

Workloads paired with sandbox execution.



Routed by...

The Model Router's decision object: task, risk, cost class, latency class, quantized variant, adapter.

Runtime optimizations — what runs where

Nine optimizations, each with a specific home in the serving stack.

OPTIMIZATION	WHERE USED
KV cache	Decode runtime
Paged KV cache	GPU memory manager
Prefix caching	Prompt-prefix reuse
Continuous batching	Serving scheduler
Chunked prefill	Long-context prompts
Speculative decoding	Predictable long responses
Quantized variants	Cost-sensitive workloads
Adapters	Repeated domain behavior
Optimized attention kernels	Long-context and throughput

Recap — deployment architecture

Zones are boundaries made physical

Seven zones isolate user access, security control, data access, model serving, sandbox, governance, and observability.



Pools follow workload shape

Seven GPU pools — small, large, reasoning, private, long-context, batch, sandbox-assisted — routed by the model router's typed decision.



Optimizations carry policy

Nine runtime optimizations, each placed deliberately — with tenant-safe caching and risk-tiered quantization keeping performance governed.

09

SECTION 9

Architecture decision records

47 ADRs in six categories, a reusable template, four worked examples, and the map connecting decisions to C4 levels.

Platform 001–005 · Governance 006–010 · RAG 011–020 · Tools · rules · agents 021–026 · Runtime 027–040 · Operations 041–047

Platform & governance ADRs

The ten decisions that define what the platform is and how it's controlled.

PLATFORM	DECISION
ADR-001	Build shared platform instead of isolated copilots
ADR-002	Apply risk-tiered architecture
ADR-003	Start with policy Q&A as first production use case
ADR-004	Prohibit direct user-to-model architecture
ADR-005	Adopt layered GenAI reference architecture

GOVERNANCE

DECISION

ADR-006

Enforce access before retrieval and tool execution

ADR-007

Implement prompt compiler as first-class component

ADR-008

Use risk tier to drive approval, logging, routing, and model choice

ADR-009

Version prompt templates and schema contracts

ADR-010

Preserve human ownership for high-impact decisions

RAG ADRs

Ten decisions that make retrieval governed knowledge, not just search.

 **ADR-011** Treat RAG corpus as governed knowledge product

 **ADR-013** Version embeddings and indexes

 **ADR-015** Use pre-retrieval ACL filtering

 **ADR-017** Use GraphRAG only for relationship-heavy questions

 **ADR-019** Separate retrieval from evidence assembly

 **ADR-012** Use content-aware chunking

 **ADR-014** Use hybrid RAG for policy and compliance search

 **ADR-016** Use temporal retrieval for effective policy handling

 **ADR-018** Use SQL RAG for structured operational facts

 **ADR-020** Add answerability gate before generation

Tool, rules, workflow & agent ADRs

Six decisions that keep action and decisioning outside the model.



ADR-021

Use governed tool gateway



ADR-022

Keep deterministic decisions outside the LLM



ADR-023

Require structured output for workflow actions



ADR-024

Execute generated code only in sandbox



ADR-025

Use bounded agents only for investigation workflows



ADR-026

Compose policy, tools, rules, and approval for fee-waiver workflow



ADR-026 is the composition proof

The fee-waiver flow from Section 7 is itself a recorded decision: policy retrieval + tool lookups + rules decision + human approval, composed in that order. Use cases are architected, not improvised.

Runtime ADRs

Fourteen decisions on serving performance, capacity, and cost.

- **ADR-027** Track prefill and decode metrics separately
- **ADR-028** Treat KV cache as first-class capacity constraint
- **ADR-029** Use paged KV cache for high-concurrency serving
- **ADR-030** Use prefix caching with tenant-safe keys
- **ADR-031** Use semantic, retrieval, and response caching with freshness and entitlement constraints
- **ADR-032** Use optimized attention kernels for supported models
- **ADR-033** Use continuous batching for shared GPU serving
- **ADR-034** Support chunked prefill for long-context workloads
- **ADR-035** Route requests by task and risk
- **ADR-036** Use adapters for repeated behavior specialization
- **ADR-037** Apply quantization by workload risk tier
- **ADR-038** Enable speculative decoding only for suitable workloads
- **ADR-039** Choose serving topology by model architecture
- **ADR-040** Use token-budget-aware serving pools

Operations ADRs

Seven decisions that keep the platform runnable, releasable, and reversible.



ADR-041

Implement multi-tenancy as platform capability



ADR-042

Enforce full-stack GenAI security controls



ADR-043

Require eval gate for every GenAI release



ADR-044

Implement end-to-end GenAI tracing



ADR-045

Define rollback units for every GenAI artifact



ADR-046

Scale evidence before autonomy



ADR-047

Maintain architecture-to-problem traceability

Scale evidence before autonomy. ADR-046
— the phasing principle in four words

The ADR template

Seventeen fields — enough rigor for a bank, light enough to actually write.



Identity

ADR ID · Title · Status · Date · Owner



Placement

C4 level — which zoom level the decision affects



Reasoning

Context · Decision · Alternatives considered · Consequences



Accountability

Risks · Controls · Operational impact · Security impact · Cost impact · Rollback · Review date



The two fields that make it enterprise-grade

Rollback — no decision is accepted without a way out. Review date — decisions expire and get re-examined; nothing is forever.

ADR-015 — Use pre-retrieval ACL filtering

The security decision that keeps unauthorized data out of model context entirely.



Status / C4 level

Accepted · Component



Context

Users have different permissions across policy documents, customer records, and operational data.



Decision

Retrieval must be scoped to entitled sources before any document enters the model context.



Alternatives considered

Retrieve everything and filter final answer



Retrieve everything and redact output

Apply ACL before retrieval ← chosen



Reason

Unauthorized data must not influence model generation.



Consequence / Rollback

Requires synchronized metadata and entitlement mapping. Rollback: disable corpus access where ACL cannot be verified.

ADR-022 — Keep deterministic decisions outside the LLM

The accountability decision behind "rules decide, models explain".



Status / C4 level

Accepted · Container / Component



Context

Fee waiver, eligibility, approval, thresholds, and policy exceptions must be consistent and auditable.



Decision

Rules engine owns deterministic decisions.
The LLM only explains rule outcomes.



Alternatives considered

Let LLM decide eligibility



Ask LLM to reason over policy

Use rules engine and let LLM explain ← chosen



Reason

Deterministic decisions need repeatability, audit, and rule versioning.



Rollback

Disable AI recommendation and fall back to existing manual rules workflow.

ADR-029 & ADR-035 — runtime decisions, same rigor

Performance choices get the full template treatment too.



ADR-029 · Use paged KV cache

Status: Accepted · C4 level: Component / Deployment Context: Variable prompt and output lengths cause KV memory fragmentation. Alternatives: static contiguous KV allocation · reduce maximum context length · paged KV allocation ← chosen Reason: Improves memory utilization and concurrency. Consequence: runtime complexity increases. Rollback: Route long-context workloads to lower-concurrency serving pool.



ADR-035 · Route requests by task and risk

Status: Accepted · C4 level: Container / Component Context: Not every task requires the largest or most expensive model. Alternatives: single default model · manual application-level selection · central model router ← chosen Reason: Improves cost, quality, and governance. Rollback: Route all production traffic to approved default model.

Master ADR map — decisions across the architecture

Every ADR anchors to a layer of the design; every layer is decision-covered.

DECISION LAYER	ADRS
Context decisions	ADR-001 shared platform · ADR-003 first use case · ADR-004 no direct user-to-model · ADR-047 architecture-to-problem traceability
Container decisions	ADR-005 layered architecture · ADR-007 prompt compiler · ADR-021 tool gateway · ADR-022 rules outside LLM · ADR-025 bounded agents · ADR-035 model router
Component decisions	ADR-006 access first · ADR-012 chunking · ADR-014–020 retrieval design · ADR-023 structured output · ADR-024 sandbox
Runtime decisions	ADR-027–034, 036–040 — KV, caching, batching, quantization, adapters, topology, pools
Security decisions	ADR-002 risk tiers · ADR-008 risk-driven behavior · ADR-015 ACL filtering · ADR-030/031 safe caching · ADR-042 full-stack controls
Deployment decisions	ADR-029 paged KV · ADR-037 quantization by tier · ADR-039 topology · ADR-040 pools
Operations decisions	ADR-041 multi-tenancy · ADR-043 eval gate · ADR-044 tracing · ADR-045 rollback units · ADR-046 evidence before autonomy · ADR-009/013 versioning

Activity 9 — Write ADRs for key trade-offs

ACTIVITY 9 · 40 MINUTES

Goal

- Produce your team's ADR register for the decisions that matter most.

Instructions

- Write ADRs covering: RAG choice, ACL filtering, temporal retrieval, tool gateway, rules engine, human approval, structured output, model router, KV cache, paged KV, eval gate, observability, rollback.
- Use the full template — especially alternatives, risks, controls, and rollback.
- Tag each ADR with its C4 level.

Expected output

- An ADR register (aim for 8–13 ADRs) with a trade-off matrix.

Evaluation criteria

Real alternatives, honestly weighed

Every ADR has a rollback

Risks paired with controls

C4 level assigned

Facilitator notes

- Reject ADRs with strawman alternatives ("do nothing") — demand the seductive wrong option, like "let the LLM decide eligibility". That's where learning happens.

Recap — architecture decision records

47 decisions, six layers

Platform, governance, RAG, tools/rules/agents, runtime, operations — the complete decision story of the platform.



The template enforces honesty

Alternatives, risks, controls, impacts, rollback, review date — decisions are defensible and reversible, never vibes.



Decisions anchor to C4

The master map ties every ADR to context, container, component, runtime, security, deployment, or operations — full traceability.

Workshop stages 1–4 — framing to components

Stage 1 · Business framing

Define: business problem, users, use cases, risk tier, non-functional requirements, success metrics.

Output: business problem canvas · risk-tier matrix · success metrics.



Stage 2 · C4 context diagram

Define: system boundary, users, external systems, data sources, model systems, audit stakeholders. Output: C4 Level 1 context diagram.



Stage 3 · C4 container diagram

Define: major deployable containers, responsibilities, interactions, security boundaries, data boundaries. Output: C4 Level 2 container diagram.



Stage 4 · C4 component diagram

Define components inside: AI gateway, task router, prompt compiler, RAG platform, tool gateway, model router, LLM runtime, validation service, observability service. Output: C4 Level 3 component diagrams.

Workshop stages 5–8 — contracts to defense

Stage 5 · Code-level contracts

Define contracts for: request, access context, risk classification, orchestration plan, chunk metadata, retrieval result, tool call, prompt package, validation result, evidence package, runtime trace.

Output: C4 Level 4 code contracts.



Stage 6 · ADR register

Write ADRs for: RAG choice, ACL filtering, temporal retrieval, tool gateway, rules engine, human approval, structured output, model router, KV cache, paged KV, eval gate, observability, rollback. Output: ADR register.



Stage 7 · Deployment and operations

Define: deployment zones, model-serving pools, runtime optimizations, observability, evals, incident response, rollback, FinOps. Output: deployment architecture · operations runbook.



Stage 8 · Final architecture defense

Defend: why it is safe, scalable, auditable, realistic — what trade-offs were made, what is built first, what is deferred. Output: final architecture defense.

Common anti-patterns to catch

Sixteen failure patterns — every one maps to a control this design already has.

direct user-to-LLM design · RAG without access control · RAG without metadata · RAG without citation validation · single vector search for every problem · LLM deciding deterministic business rules · tool calling without policy gateway · write tools without idempotency · agents without step limits · no human approval for high-impact action · no eval suite · no rollback strategy · no model routing · no token and KV capacity planning · no observability · no ADRs



Facilitator use

Run this list against every team's design during stage reviews. Each hit is a teachable moment: name the anti-pattern, then name the ADR or component that prevents it.

Activity 10 — Design the evaluation and observability plan

ACTIVITY 10 · 30 MINUTES

Goal

- Specify how quality is proven before release and watched after.

Instructions

- Define golden datasets for the policy Q&A and fee-waiver flows.
- Choose eval runners: RAG, model, tool, agent, security — and what each gates.
- Define the regression gate policy: what blocks a release?
- Pick production monitors: traces, cost per tenant, cache hits, retrieval quality, approval analytics, incidents.
- Define incident response and rollback per artifact type.

Expected output

- An evaluation plan + observability plan + incident and rollback plan (deliverables 12–14).

Evaluation criteria

- Eval gate applies to every release type (model, prompt, index, tool)
- Monitors cover quality, cost, and safety — not just uptime
- Rollback units named per artifact (ADR-045)

Facilitator notes

- Push for specificity: "monitor quality" fails; "alert when citation-validity rate drops below threshold on sampled production traffic" passes.

Activity 11 — Defend the final architecture

ACTIVITY 11 · 45 MINUTES (10 PER TEAM + PANEL Q&A)

Goal

- Present and defend the complete architecture against a critical panel.

Instructions

- Present: context → containers → one component deep-dive → contracts → deployment → ADR highlights.
- Answer the seven defense questions: why safe, why scalable, why auditable, why realistic, what trade-offs, what first, what deferred.
- Panel (facilitators or peer tables) challenges two decisions.

Expected output

- Final architecture defense (deliverable 17).

Evaluation criteria

- Claims trace to diagrams, contracts, and ADRs — not assertion

Trade-offs acknowledged honestly

- Phasing is realistic (evidence before autonomy)

Facilitator notes

- Best challenge questions: "show me the rollback for that" and "which anti-pattern is closest to your design?" Score with the rubric live.

11

SECTION 11

Final synthesis

The completeness check, the final corrected architecture statement, the integrated stack, the roadmap, and what to take home.

Sanity check · Final architecture statement · Why the design works · Integrated architecture · Deployment roadmap · Takeaways · Glossary · Coverage audit

Final architecture statement

Aarohan Bank's internal GenAI platform is not a chatbot. It is a C4-designed enterprise AI operating system.

C4 Context

Connects internal users, policy repositories, banking systems, data platforms, model runtimes, workflows, audit, and observability.

C4 Container

Separates the system into deployable services — portal, identity & risk, gateway, router, compiler, RAG, SQL RAG, GraphRAG, tools, rules, approval, agents, sandbox, model router, LLM runtime, validation, evaluation, observability, audit.

C4 Component

Decomposes each container into concrete parts — ACL filters, temporal retrievers, rerankers, context packers, token-budget managers, tool policy engines, schema validators, KV cache managers, schedulers, evidence packagers.

C4 Code

Defines contracts for requests, access context, risk, orchestration plans, chunks, retrieval results, tool calls, prompt packages, rule decisions, validation results, evidence packages, runtime traces.

Why the production design works

Fourteen controls — each sentence names an owner for a class of risk.

- Governance controls risk.
- RAG controls evidence.
- Model router controls model choice.
- KV cache and batching control latency and memory.
- Rules control deterministic decisions.
- Validation controls output safety.
- Observability controls operations.
- Identity controls access.
- Prompt compiler controls context.
- LLM runtime controls inference.
- Tools control live system access.
- Humans control accountability.
- Evals control release quality.
- ADRs control architectural discipline.

The final integrated architecture

Everything on one slide — the governed spine, the knowledge & action fan-out, and the assurance floor.

User → Portal → Identity · Entitlement · Consent · Risk → AI Gateway → Task Router → Prompt Compiler → Context Orchestrator

RAG Platform · GraphRAG · SQL RAG · Tool Gateway · Rules Integration · Cache
Agent Runtime / Sandbox

Model Router → LLM Serving Runtime tokenize · prefill · KV · decode · batch · stream → Validation & Evidence → Human Approval → Output / Action

Audit Store · Observability & FinOps · Evaluation & release gates · Registries —
prompt · model · adapter · corpus · schema · tool · index

Deployment roadmap — five phases revisited

Now you can see what each phase actually deploys.



Phase 1 · Governed policy Q&A

Portal, identity & risk, gateway, task router, prompt compiler, RAG platform + ingestion, model router (basic), runtime, validation, audit — the full spine, one use case



Phase 2 · RAG + tools + approval

Tool gateway, rules integration, workflow & human approval — the fee-waiver flow goes live



Phase 3 · SQL RAG + GraphRAG + workflows

Semantic-layer analytics, knowledge graph, structured workflow actions



Phase 4 · Agents + optimized runtime

Bounded agent runtime, sandbox, paged KV, prefix caching, continuous batching, speculative decoding



Phase 5 · Enterprise scale

Full model routing, observability & FinOps maturity, eval automation, multi-tenant scale

Scale evidence before autonomy. ADR-046 — why the phases are ordered this way

What to take home

1 · Platforms beat pilots

One governed platform with shared controls for identity, entitlement, RAG, tools, routing, workflow, evaluation, observability, cost, and audit — that's the ambition worth funding.



2 · The model is one box of thirteen

Production readiness lives in the stages around the LLM: access first, evidence gated, rules deciding, humans attesting, everything audited.



3 · C4 + ADR is the method

Four zoom levels give the design its structure; forty-seven recorded decisions give it discipline, reversibility, and traceability.



4 · Phase by evidence

Start with governed policy Q&A, add tools and approval, then structured retrieval, then bounded agents — scaling autonomy only as evidence infrastructure matures.

This is realistic, workshop-ready, C4-ready, ADR-ready, and deployable as a complete internal GenAI system design case study. Closing line of the source design

Glossary

C4

Four-level architecture model: Context, Container, Component, Code

ADR

Architecture Decision Record — context, decision, alternatives, consequences, rollback

RAG

Retrieval-augmented generation — grounding answers in retrieved documents

Hybrid retrieval

Keyword + vector search combined by rank fusion

ACL-aware retrieval

Entitlement filtering applied before retrieval

Temporal retrieval

Selecting currently effective document versions

GraphRAG

Retrieval over an entity-relationship knowledge graph

SQL RAG

Governed natural-language analytics via a semantic layer

Corrective RAG

Retrieval strategy that re-retrieves when evidence is weak

Answerability gate

Check that evidence suffices before generation proceeds

DLP

Data loss prevention — scanning for sensitive data and secrets

KV cache

Stored attention keys/values reused during decode

Paged KV cache

Block-based KV memory management against fragmentation

Prefix caching

Reusing computation for shared prompt prefixes

Prefill / decode

Processing the prompt vs. generating tokens — tracked separately

TTFT / TPOT

Time to first token · time per output token

Continuous batching

Scheduler keeps GPU batches full as requests arrive/finish

Chunked prefill

Splitting long prompt processing into chunks

Speculative decoding

Draft model proposes, verifier model confirms tokens

Quantization

Reduced-precision model variants (int8, int4, fp8) for cost

Adapter / LoRA

Lightweight model specialization loaded at runtime

Evidence package

Immutable record of prompt, sources, versions, decisions, attestation